Step Ahead, Distill Back: One-Step RL-Guided On-Policy Distillation

Zhou Jiang[1] Ruotian Peng[2] Haoquan Zhang[3] Yandong Wen[2] Zhen Liu[1]*

[1]The Chinese University of Hong Kong, Shenzhen
[2]Westlake University [3]The Chinese University of Hong Kong

Abstract

On-policy distillation (OPD) offers dense supervision for reasoning models, but its use with reward-trained experts commonly follows a train-then-distill workflow. We show that effective guidance can instead come from a teacher only one reward-gradient step ahead of the student. Our method, On-Policy Gradient Distillation (OPGD), rebuilds this teacher from the current actor at every iteration, allowing it to inherit accumulated learning while adapting to fresh verified outcomes. The teacher–actor log ratio isolates the preferences induced by this adaptation and supplies signed token advantages for OPD on the same rollouts. A first-order analysis connects these preferences to interactions between token score gradients, explaining how response-level feedback is redistributed within and across responses. Distillation reweights the outcome gradient through the policy's local geometry while preserving nonnegative first-order alignment. On Qwen3-4B, OPGD improves mean Pass@1 over GRPO by 9.2 percentage points across four mathematical benchmarks at a common 100-update budget. Separate training on coding prompts also yields faster learning and higher accuracy across three code-generation benchmarks.

1 Introduction

Reinforcement learning with verifiable rewards (RLVR) has driven substantial progress in mathematical reasoning and code generation [9, 24]. Optimizing verifiable outcomes allows models to develop reasoning behavior from their own sampled trajectories without annotations of intermediate steps. The resulting policies can also serve as teachers, transferring the benefits of reward training to other models through distillation [9]. Access to these experts' conditional distributions enables token-level supervision, carrying their reward-adapted preferences into the student's learning objective [29].

On-policy distillation (OPD) uses these conditional preferences on student-generated trajectories, keeping supervision aligned with the student's evolving rollout distribution [1, 7]. In a train-then-distill workflow, this feedback is supplied by an expert whose reward adaptation is already complete. An alternative is to obtain teacher–student preferences from incremental reward adaptation of the student itself, refreshing the teacher throughout learning. This raises the question of whether a teacher only one reward-gradient step ahead can already provide effective distillation supervision.

We propose On-Policy Gradient Distillation (OPGD), which couples reward adaptation and OPD through a teacher only one outcome-based policy-gradient step ahead of the actor. Reconstructed from the current actor at every iteration, the teacher inherits its accumulated learning, allowing a single adaptation step to supply fresh guidance throughout training. The teacher–actor log ratio isolates the preferences induced by the latest verified outcomes and serves as a signed token advantage for OPD on the same rollouts.

*Corresponding Author.

Distilling these preferences transforms the outcome gradient according to the actor's local geometry. Our first-order analysis identifies how token score similarities redistribute outcome feedback across contexts, yielding different supervision within a response and transferring feedback between responses. Fitting these preferences determines a new update that retains nonnegative first-order alignment with the outcome gradient. Experiments on Qwen3-4B show that OPGD improves mean Pass@1 over GRPO by 9.2 percentage points across four mathematical benchmarks at the same 100-update budget. Separate code-generation experiments likewise show faster learning and higher accuracy.

In summary, our contributions are:

- We introduce a one-step teacher construction that converts outcome supervision into contextual OPD advantages, enabling distillation within the actor's reward training without an external expert.
- We derive a local kernel interpretation of these advantages and their induced update, explaining cross-response credit transfer and nonnegative first-order alignment with the outcome gradient.
- We validate OPGD on four mathematical reasoning benchmarks with Qwen3-4B and on three code-generation benchmarks using separate coding training, demonstrating faster learning and higher accuracy than GRPO.

2 Related Work

On-policy distillation and reward adaptation. Training on student-generated trajectories addresses the distribution mismatch that arises when sequence distillation relies on teacher-generated responses [1]. The teacher then supervises prefixes encountered by the student, while the choice of divergence determines how its preferences enter the update. MiniLLM develops reverse-KL distillation through policy-gradient optimization [7]. For reasoning models, G-OPD uses an immediate token-level log-ratio advantage and interprets distillation as optimizing the teacher's implicit reward [29]. This formulation connects policy matching to reward maximization and permits reward extrapolation, allowing a student to learn beyond the teacher's performance rather than treating imitation as the endpoint. OPD^2 focuses the reward on changes acquired during post-training by comparing the teacher with its own pretrained base [11]. It centers these delta advantages under the student distribution and retains them where their signs agree with the ordinary teacher–student advantages. The reference therefore isolates what the teacher acquired, while the student still determines which changes enter distillation. Empirical studies likewise find that teacher accuracy alone does not determine transfer: compatibility with the student and capabilities acquired through further teacher training affect the usefulness of its feedback [17].

Outcome supervision and contextual credit. Policy ratios also connect distillation to dense feedback learned from coarse supervision. DPO parameterizes preference-derived rewards through a policy–reference log ratio [21], and its token-MDP formulation relates these ratios to sequential credit assignment [22]. With outcome labels, Yuan et al. [31] show that the same reward parameterization yields an implicit process reward model without step annotations. PRIME makes this construction part of online RL, updating the implicit reward model on verified rollouts and combining returns from its token rewards with the outcome advantage [4]. Process supervision instead learns step correctness directly from annotations [18], whereas VinePPO estimates prefix values by sampling additional continuations, replacing learned value predictions with Monte Carlo estimates [15]. With only the original rollouts, KTAE supplements the response advantage with token-type associations estimated from successful and unsuccessful responses [26], while entropy-based selection concentrates updates on uncertain positions where alternative continuations remain plausible [27]. OPGD derives its signed coefficients from the actor's response to the verified outcomes, making the feedback sensitive to each token's prefix without a persistently trained process model or additional continuation sampling.

Self-distillation and teacher construction. Teacher-free formulations replace external feedback with advantages based on the student's probabilities and entropy [5]. Context-based self-distillation instead gives the teacher information unavailable in the student's original prompt. Reference solutions [32], successful attempts, and environment feedback [12] can improve the model's conditional predictions, which are distilled back into its behavior under the original prompt. RISE obtains a teacher through reward adaptation, extrapolating an RLVR-updated policy relative to an earlier or averaged anchor, then distilling the extrapolated policy into the actor after its RL update [16]. RISE therefore retains the RLVR step and adds distillation, whereas OPGD uses reward adaptation only to construct actor-relative token advantages for its retained distillation update.

3 Preliminaries

Let $\pi_\theta(x \mid q)$ be an autoregressive policy and $R(q, x)$ a terminal verifier reward. For response x_i, the token prefix is $c_{it} = (q_i, x_{i,<t})$. At iteration k, rollouts are sampled from π_{θ_k}. Their prefixes remain fixed during optimization. The valid response positions are indexed by $\mathcal{K}$, with $\mathcal{K}_i$ denoting those in response i.

3.1 Group-relative policy optimization

GRPO [24] uses G responses per prompt to compute group-normalized outcome advantages:

$$A_i = \frac{R_i - \bar{R}_{q_i}}{\sigma_{q_i} + \varepsilon}, \qquad \bar{R}_q = \frac{1}{G} \sum_{j:q_j=q} R_j. \tag{1}$$

Here $R_i = R(q_i, x_i)$, σ_q is the group reward standard deviation, and $\varepsilon > 0$. At the rollout policy, with inactive ratio clipping and no reference-policy penalty, the outcome gradient is

$$\boldsymbol{s}_{it} = \nabla_\theta \log \pi_{\theta_k}(x_{it} \mid c_{it}), \qquad g_k = \sum_{(i,t)\in\mathcal{K}} w_{it} A_i \boldsymbol{s}_{it}. \tag{2}$$

The fixed nonnegative weights w_{it} include masking, loss normalization, and any fixed importance correction. The response-level coefficient A_i is shared across positions, whereas the token scores $\boldsymbol{s}_{it}$ depend on the sampled prefixes.

3.2 On-policy distillation

OPD evaluates a teacher on student-generated prefixes [1, 7]. We use the token-local reverse-KL formulation, with the prefix distribution C_k held fixed:

$$\mathcal{L}_{\mathrm{KL}}(\theta; \phi) = \mathbb{E}_{c\sim C_k}\left[\mathrm{KL}\big(\pi_\theta(\cdot \mid c) \,\|\, \pi_\phi(\cdot \mid c)\big)\right]. \tag{3}$$

For a fixed teacher, its gradient at θ_k admits the sampled-token advantage form

$$\begin{aligned} A^{\phi}_{\mathrm{OPD}}(c, a) &= \log \pi_\phi(a \mid c) - \log \pi_{\theta_k}(a \mid c), \\ -\nabla_\theta \mathcal{L}_{\mathrm{KL}}\big|_{\theta_k} &= \mathbb{E}_{c,a}\left[A^{\phi}_{\mathrm{OPD}}(c, a) \nabla_\theta \log \pi_{\theta_k}(a \mid c)\right], \end{aligned} \tag{4}$$

with $a \sim \pi_{\theta_k}(\cdot \mid c)$. This immediate log-ratio advantage is the token-level OPD signal used by Yang et al. [29]. OPGD instantiates it with a reward-updated teacher and freezes the resulting advantages in an empirical loss on the current rollouts.

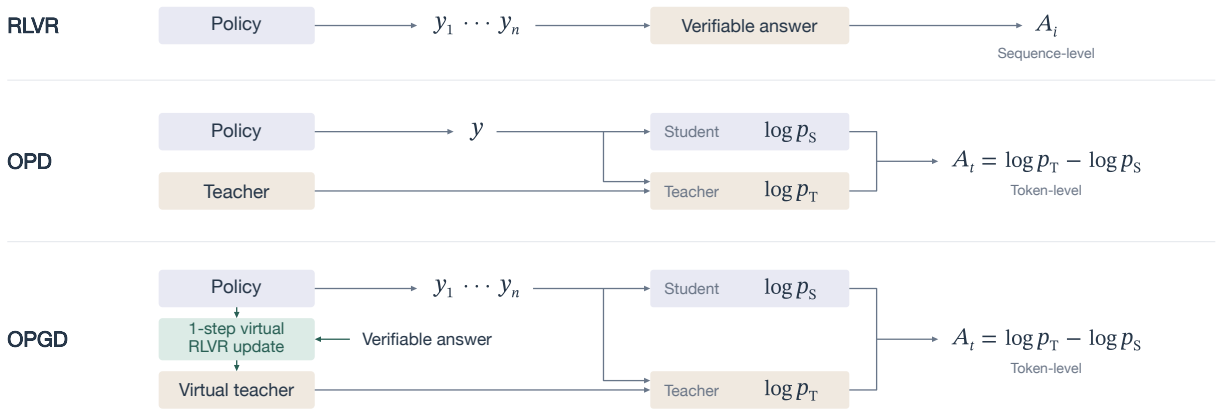


Figure 1: Comparison between RLVR, OPD, and OPGD.

4 Method

Figure 1 contrasts the three paradigms. RLVR scores a response with a sequence-level outcome advantage, OPD distills a token-level signal from a separate teacher, and OPGD keeps that token-level signal but builds the teacher from one virtual RLVR step on the actor's own verified rollouts.

4.1 Learning from a one-step teacher

Starting from the current actor θ_k, we construct its teacher with one SGD step on the outcome objective:

$$\theta_k^E = \theta_k + \alpha g_k. \tag{5}$$

The teacher is adapted on the actor's own verified rollouts, and the actor serves as both its student and its pre-adaptation reference. The OPD advantage therefore coincides with the teacher's reward-adaptation delta,

$$d_{it} = \log \pi_{\theta_k^E}(x_{it} \mid c_{it}) - \log \pi_{\theta_k}(x_{it} \mid c_{it}). \tag{6}$$

This choice of reference connects the construction to delta distillation [11]: teacher and student share their preceding training, and their log ratio isolates the preferences acquired through the current reward step. Each iteration builds a new teacher from the updated actor, so the teacher's one-step horizon incorporates the actor's accumulated learning.

We optimize the original actor with these detached advantages:

$$\mathcal{L}_{\mathrm{OPGD}}(\theta) = -\sum_{i,t} v_{it}\ \mathrm{sg}[d_{it}] \log \pi_\theta(x_{it} \mid c_{it}), \tag{7}$$

where v_{it} includes loss normalization and masking. For efficiency, teacher construction and distillation reuse the same rollout batch, avoiding additional generation (Algorithm 1). The teacher consequently depends on the same sampled contexts and actions used to estimate its distillation gradient, introducing bias relative to distillation on independent actor rollouts with the teacher held fixed.

We use plain SGD for the virtual step, consistent with its competitive performance against AdamW in RLVR [20]. This stateless update requires no separate teacher moments and avoids inheriting the actor's Adam moments, which accumulate gradients of the distillation objective.

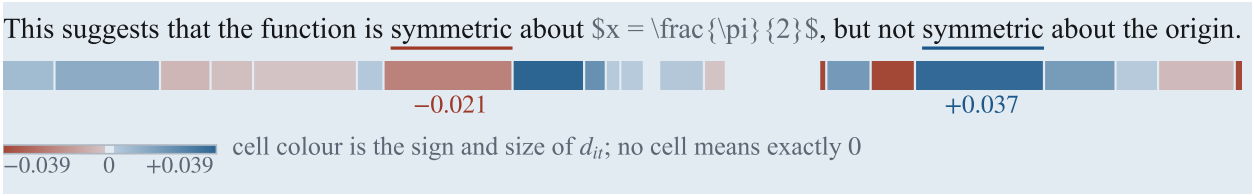


Figure 2: Contextual credit from a one-step teacher: in one zero-advantage response fragment, the two occurrences of *symmetric* receive opposite advantages.

Algorithm 1 On-Policy Gradient Distillation (OPGD)

Require: Actor θ_k, verifier R, prompt batch, group size G, lookahead rate α, retained fraction f

Collect verified trajectories

Sample G responses per prompt from π_{θ_k} and evaluate their rewards

Compute group advantages A_i and the outcome gradient g_k using Equations equation 1–equation 2

Construct the temporary teacher

Save actor parameters θ_k and sampled-token log probabilities ℓ^0_{it}

Apply the stateless SGD step $\theta \leftarrow \theta_k + \alpha g_k$

$d_{it} \leftarrow \mathrm{sg}[\log \pi_\theta(x_{it} \mid c_{it}) - \ell^0_{it}]$

Distill back into the actor

Restore $\theta \leftarrow \theta_k$ and clear temporary gradients

Retain the top fraction f of $|d_{it}|$ within each response ($f = 1$ retains all tokens)

Update the actor once using Equation equation 7 and advance its optimizer and schedule

return updated actor θ_{k+1}

Teacher strength. The lookahead rate α controls the teacher's adaptation independently of the actor learning rate. Reward extrapolation [29] combines teacher and actor logits as

$$z_{\text{ext}} = (1 + \omega)z_E - \omega z_k, \qquad \omega \geq 0. \tag{8}$$

For a one-step teacher, this is locally equivalent to using virtual learning rate $(1 + \omega)\alpha$. At the original actor, logit extrapolation scales the full-action OPD gradient exactly, without changing its direction (Appendix A.6).

4.2 NTK interpretation of contextual credit

The one-step teacher is optimized with response-level advantages, yet its OPD feedback can distinguish tokens within a response. This redistribution can be understood through the gradient interactions induced by reward adaptation. Linearizing the teacher's log-probability changes gives

$$d_{it} \approx \alpha \boldsymbol{s}_{it}^\top g_k = \alpha \sum_{j,u} w_{ju} A_j \underbrace{\boldsymbol{s}_{it}^\top \boldsymbol{s}_{ju}}_{K(it,ju)}. \tag{9}$$

Here K is the empirical NTK of sampled-token log probabilities [13], connecting the teacher's preferences to the cross-example gradient interactions studied in LLM finetuning by Ren and Sutherland [23].

The diagonal contribution has the sign of the token's own response advantage, with magnitude proportional to its squared score norm. Interactions with other tokens can reinforce or reverse this contribution, allowing a common outcome signal to induce different preferences across contexts. Different uses of the same token can therefore receive opposite coefficients, as in Figure 2.

Because these interactions extend across responses, a zero outcome advantage does not preclude a distillation signal. When $A_i = 0$, all contributions from that response vanish and its targets arise entirely from other responses. Homogeneous-outcome groups can thus receive contextual feedback from outcome differences elsewhere in the batch despite contributing no reward gradient of their own.

4.3 Distillation geometry and reward alignment

Distilling these contextual preferences can change the actor's learning direction even when the teacher takes only the original outcome-gradient step. Matching conditional predictions weights that step through the actor's sensitivity across distillation contexts. Substituting Equation equation 9 into the distillation objective gives

$$h_k = -\nabla_\theta \mathcal{L}_{\mathrm{OPGD}}\big|_{\theta_k} \approx \alpha F_k g_k, \qquad F_k = \sum_{i,t} v_{it} \boldsymbol{s}_{it} \boldsymbol{s}_{it}^\top. \tag{10}$$

The empirical score second moment F_k therefore includes contexts from zero-advantage responses that do not contribute directly to g_k.

Directions with larger eigenvalues of F_k produce larger aggregate changes in sampled-token log probabilities and receive greater relative weight. Thus the distinction between distillation and the original reward step depends on the actor's local geometry: an isotropic F_k only rescales g_k, whereas anisotropy can redirect it.

This redirection preserves nonnegative alignment with the outcome gradient to first order because F_k is positive semidefinite for nonnegative distillation weights. In particular,

$$g_k^\top h_k \approx \alpha \sum_{i,t} v_{it} (\boldsymbol{s}_{it}^\top g_k)^2 \approx \frac{1}{\alpha} \sum_{i,t} v_{it} d_{it}^2 \geq 0. \tag{11}$$

Since each position contributes in proportion to $v_{it} d_{it}^2$, this alignment also provides a criterion for selecting which preferences to distill. With a fixed retention count and equal weights within each response, selecting the largest $|d_{it}|$ preserves the greatest local alignment. This motivates the magnitude-selection variant evaluated in Section 5.4, with the derivation and weighting conditions given in Appendix A.

5 Experiments

5.1 Experimental setup

We fine-tune Qwen3-4B [28] on DeepMath-103K prompts [10] with binary verifier rewards. Each batch contains 128 prompts and eight responses per prompt, sampled at temperature 1.0 and top-p = 1.0, with a 16,384-token response limit. The mathematical training runs share their initialization and use full-parameter AdamW with learning rate 10^{-6}, no reference-policy KL penalty, and no entropy bonus. OPGD constructs its temporary teacher with an SGD step of size $\alpha = 0.1$. The remaining settings are given in Appendix D.

The main comparison includes GRPO, DAPO [30], entropy-masked GRPO, full-target OPGD, and responsewise top-10% OPGD. Entropy-masked GRPO retains the highest-entropy 20% of tokens in each response in its policy-gradient loss, with the other baseline settings unchanged. The GRPO run is evaluated within 100- and 200-update budgets, and the other configurations use a 100-update budget. Published 500-update GRPO weights from Yang et al. [29] provide an external accuracy reference. Evaluation uses 32 completions per problem on AIME'24, AIME'25, HMMT-Feb'25, and HMMT-Nov'25. Following prior work [5], each run is reported at its best checkpoint within its budget (Appendix D). Code-generation results use separate runs trained on Eurus-RL-Code prompts [4].

5.2 Learning progress across tasks

Table 1: Mathematical reasoning performance at checkpoints selected within each training-update budget. The mean weights the four benchmarks equally. The 500-update GRPO reference uses published weights from Yang et al. [29].

Metric	Method	Budget	AIME24	AIME25	HMMT-Feb25	HMMT-Nov25	Mean
Pass@1	Qwen3-4B	–	0.2229	0.2073	0.1094	0.0875	0.1568
	GRPO	100	0.4917	0.4281	0.2688	0.3135	0.3755
	GRPO (entropy 20%)	100	0.5458	0.4760	0.2833	0.3385	0.4109
	DAPO	100	0.5531	0.5240	0.2844	0.3552	0.4292
	GRPO	200	0.5844	0.5115	0.2969	0.3469	0.4349
	GRPO (external)	500	0.5823	**0.5604**	0.3219	0.3833	0.4620
	OPGD (full)	100	0.6063	0.5521	0.3302	0.3823	0.4677
	OPGD (top 10%)	100	**0.6667**	0.5448	**0.3542**	**0.3906**	**0.4891**
Pass@8	Qwen3-4B	–	0.4333	0.3339	0.2323	0.2745	0.3185
	GRPO	100	0.7843	0.6697	0.4483	0.5199	0.6056
	GRPO (entropy 20%)	100	0.7784	0.6787	0.4755	0.5398	0.6181
	DAPO	100	0.7736	0.6902	0.4649	0.5327	0.6154
	GRPO	200	0.7853	0.6804	0.5106	0.5452	0.6304
	GRPO (external)	500	0.7846	0.6925	0.4799	0.5409	0.6245
	OPGD (full)	100	0.7968	0.7212	0.4899	**0.5941**	0.6505
	OPGD (top 10%)	100	**0.8308**	**0.7354**	**0.5329**	0.5885	**0.6719**

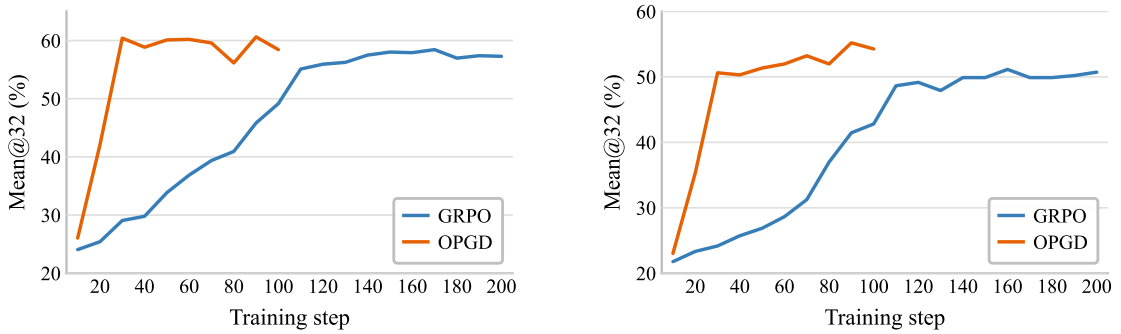
Within the same 100-update budget, full-target OPGD surpasses GRPO across all four mathematical benchmarks and all three Pass@k metrics, with mean scores exceeding those of both the 200-update run and the published 500-update reference (Table 1and Appendix Table 6). The advantage develops early: on AIME, OPGD reaches an accuracy plateau within the first few dozen updates while GRPO improves gradually (Figure 3a–b). That plateau persists as mean response length contracts by more than half from its peak (Appendix E). Selecting the strongest 10% of targets further improves the mean benchmark scores.

Separate training on coding prompts yields the same early advantage. On HumanEval+, MBPP+ [19], and LiveCodeBench v6 [14], OPGD reaches GRPO's update-100 accuracy within 30–40 updates and remains ahead at every shared evaluation (Figure 3c–e). Appendix F gives the code-generation results in detail.

Cross-domain evaluation with Qwen3-8B. To test whether the improvement transfers beyond the mathematical training task, we also evaluate Qwen3-8B after the same 100-update budget for GRPO and OPGD. The four mathematical benchmarks measure in-domain reasoning, while HumanEval, HumanEval+, MBPP, MBPP+, and LiveCodeBench v6 serve as out-of-domain code-generation tests. Table 2 reports Pass@1 and Pass@4 in the main text; the complete Pass@1/2/4/8 breakdown appears in Appendix B; all entries are percentages.

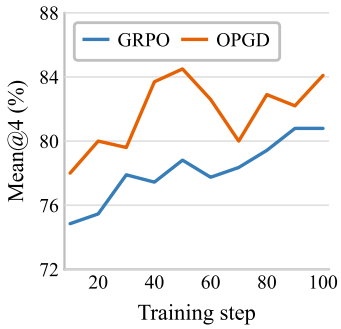
Table 2: Qwen3-8B evaluation after 100 updates for both GRPO and OPGD. The table reports Pass@1 and Pass@4. The mathematical benchmarks are in-domain, while the code benchmarks test out-of-domain transfer. Values are percentages; bold marks the best value in each column and blue rows identify OPGD.

Metric	Method	Math task					Code task					
		AIME24	AIME25	HMMT-Feb	HMMT-Nov	avg	HumanEval	HumanEval+	MBPP	MBPP+	LCB v6	avg
Pass@1	Base	27.2	21.7	10.1	9.2	17.0	88.0	81.3	80.8	69.2	17.0	67.2
	GRPO	54.0	41.0	26.0	35.6	39.2	85.1	80.5	81.5	70.8	17.4	67.1
	OPGD	**67.2**	**51.2**	**31.9**	**44.1**	**48.6**	**88.7**	**82.2**	**84.8**	**73.3**	**18.1**	**69.4**
Pass@4	Base	43.5	31.6	16.9	21.7	28.4	**95.7**	89.6	89.4	77.8	**22.9**	75.1
	GRPO	74.8	58.8	40.2	50.9	56.2	90.2	86.6	89.2	77.8	**22.9**	73.3
	OPGD	**79.8**	**69.4**	**48.8**	**58.6**	**64.1**	94.5	**90.9**	**90.0**	**79.4**	**22.9**	**75.5**

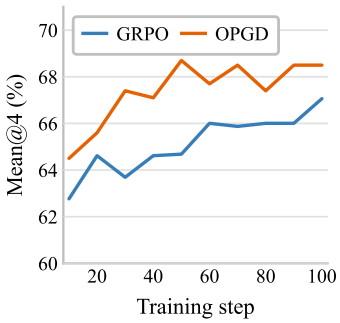


(a) AIME'24

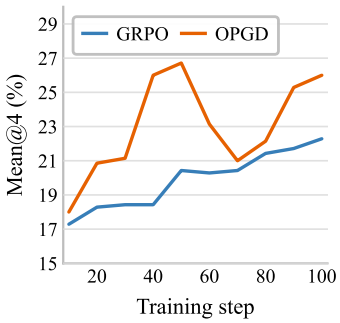
(b) AIME'25



(c) HumanEval+



(d) MBPP+



(e) LiveCodeBench v6

Figure 3: **Learning progress in mathematical reasoning and code generation.** Top: mean sample correctness over 32 completions per problem on AIME, with full-target OPGD trained for 100 updates and GRPO for 200. Bottom: code-generation Pass@1 from separate runs trained on coding prompts over the first 100 updates, using four completions per problem.

Computational cost. The faster progress per update comes with additional teacher computation. An OPGD update takes about 1.5 times as long as GRPO in the measured profile, with overhead concentrated in the virtual reward backward pass and teacher scoring (Figure 4). Target selection retains complete-sequence scoring and contextual backpropagation, so this overhead does not fall proportionally with the selected fraction. Total training time also depends on the evolving response lengths.

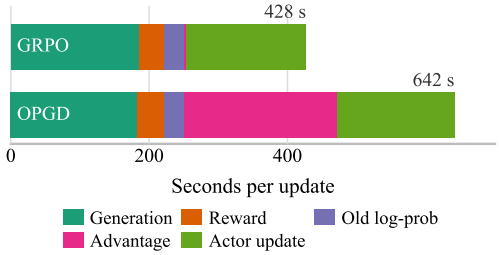


Figure 4: The first-update runtime.

5.3 Feedback from a one-step teacher

Nearly half the responses in the first diagnostic batch carry zero group advantage and contribute nothing to the outcome gradient, yet most of their teacher targets are nonzero, and the mixture of signs recurs across positive-, negative-, and zero-advantage responses (Figures 14 and 15). The sign follows the score interactions of Equation 9: a token's coefficient sums the batch outcome signal over its similarities with the other sampled tokens, so contexts drawing on different parts of the batch move oppositely.

Table 3: **Training cost against accuracy.** Wall clock per run, rescaled to 8×A800 by first-step latency, with the four-benchmark mean Pass@k scores from Table 1.

Method	Budget	Wall clock (h)	Pass@1	Pass@4	Pass@8
GRPO	100	15.88	0.3755	0.5515	0.6056
GRPO	200	56.58	0.4349	0.5799	0.6304
DAPO	100	40.00	0.4292	0.5698	0.6154
OPGD	100	31.31	**0.4677**	**0.6132**	**0.6505**

Magnitude instead tracks token uncertainty, which cannot separate the two signs: positions of comparable entropy split into both (Figure 9). Entropy-masked GRPO improves over the GRPO baseline through such a criterion, keeping the response-wide advantage at the positions it selects [27]; full-target OPGD reaches higher means on all three metrics (Table 1) with a criterion that retains the sign.

Gradient structure. At the same actor and rollout batch, matching these preferences increases the concentration of singular-value energy in the parameter-gradient matrices before token selection (Figure 5). The median share of energy in the leading sixteen singular directions rises from 55.5% to 77.9%, with increases in nearly all matrices. Estimated spectral entropy also decreases, most strongly in later layers and the embedding. Because these spectra are normalized, the change reflects redistribution of gradient energy beyond a common rescaling of the outcome update (Appendix I).

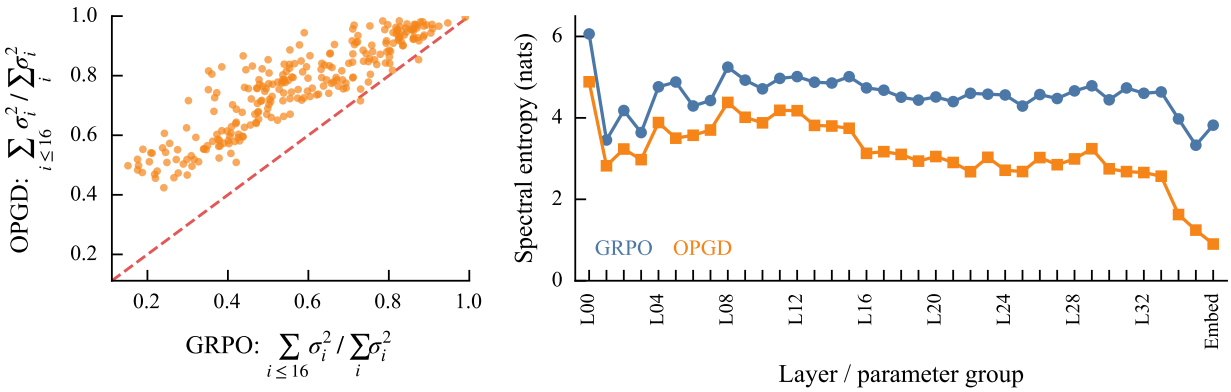


Figure 5: **Gradient structure under full-target distillation**, measured from the same actor and batch. Left: leading-16 energy shares for 253 gradient matrices, normalized by the exact squared Frobenius norm. Right: layer-group spectral-entropy proxy with a uniform unresolved tail.

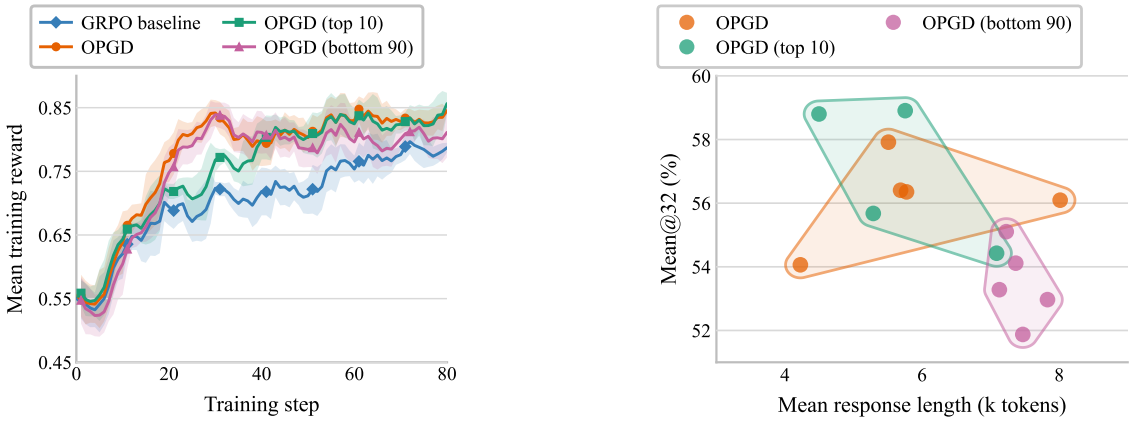
Table 4: Mathematical reasoning with complementary target subsets. Tokens are ranked within each response by absolute OPD advantage. Entries are percentages, and the mean weights the four benchmarks equally.

	Pass@1		Pass@4		Pass@8	
Benchmark	Top 10%	Bottom 90%	Top 10%	Bottom 90%	Top 10%	Bottom 90%
AIME24	66.67	59.48	81.22	76.67	83.08	80.47
AIME25	54.48	50.10	69.21	63.21	73.54	68.30
HMMT-Feb25	35.42	34.27	47.93	46.27	53.29	49.65
HMMT-Nov25	39.06	38.54	53.92	52.45	58.85	57.29
Mean	48.91	45.60	63.07	59.65	67.19	63.93

5.4 Effect of target selection

The full-target gains leave open how much of the teacher's feedback is needed for effective distillation. On the first diagnostic batch, the globally largest 10% of targets carry 76.9% of absolute mass, with approximately balanced positive and negative contributions (Appendix G). We compare retaining the largest 10% within each response with retaining the complementary 90%, constructing both teachers from the complete batch before selection.

Distilling the top 10% preserves rapid learning and approaches the full-target reward curve; the bottom 90% rises faster early on but converges in reward (Figure 6a). The top 10% still achieves higher scores across all benchmarks and sampling budgets (Table 4), and at later checkpoints combines higher accuracy with shorter responses (Figure 6b).



(a) Training reward

(b) Accuracy and response length

Figure 6: **Complementary target subsets.** Left: five-update reward averages with $\pm 1\sigma$ bands of raw rewards. Right: AIME Mean@32 (%) against mean response length at checkpoints after update 50, with each run's checkpoints enclosed by the outline of their convex hull. Each run keeps its colour across the two panels.

6 Discussions

An implicit preference over policies. Similar training rewards accompany different benchmark accuracy and response lengths across target subsets. This suggests that the choice of distillation contexts influences which behavioral solutions are reached during reward optimization. One possible explanation is an implicit optimization bias induced by target matching, connecting OPGD to the role of optimization geometry in solution selection [8]. Here the geometry depends on the sampled contexts and, under screening, on the teacher's reward-directed preferences. It offers a route for favoring particular policy changes without adding a reference-policy KL penalty.

Designing the teacher and matching objective jointly. Bootstrapped meta-learning treats target construction and the matching metric as separate design choices [6]. For OPGD the interaction is concrete: under a shared full-target score moment, inverse-score preconditioning in the virtual step would locally cancel the reweighting from conditional-policy matching (Appendix A.4), so compatibility with the matching objective, not standalone reward, becomes a criterion for teacher design.

Batch construction and the learning rule. Pooling rollouts into one teacher generally differs from constructing separate teachers for rollout partitions and averaging the updates distilled on each partition. With full targets and consistent normalization, the pooled update includes interactions between one partition's reward gradient and another's distillation contexts under the local approximation. Separate teachers omit these cross-partition terms (Appendix A.2).

7 Conclusion

We showed that a teacher only one reward-gradient step ahead can sustain effective on-policy distillation throughout outcome-based training. OPGD turns incremental reward adaptation into contextual supervision, with feedback extending across responses and a retained gradient shaped by the actor's local geometry. On Qwen3-4B, this construction accelerates learning in separate mathematical reasoning and code-generation experiments. The results support continually renewing the teacher alongside the actor, making distillation part of policy optimization without a separate expert-training stage.

AI Use Statement

AI assistants were used during the preparation of this manuscript to assist with limited aspects of the writing process, including reorganizing LaTeX source files, improving the clarity and readability of the prose, and refining the presentation of technical descriptions. All mathematical formulations, experimental results, analyses, and scientific claims were independently verified by the authors. The authors have carefully reviewed and revised all AI-assisted edits and remain fully responsible for the accuracy, integrity, and final content of the submitted manuscript.

References

[1] Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes. In *ICLR*, 2024.

[2] Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998. doi: 10.1162/089976698300017746.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[4] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, Jiarui Yuan, Huayu Chen, Kaiyan Zhang, Xingtai Lv, Shuo Wang, Yuan Yao, Xu Han, Hao Peng, Yu Cheng, Zhiyuan Liu, Maosong Sun, Bowen Zhou, and Ning Ding. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

[5] Yi Ding and Ruqi Zhang. Does on-policy distillation really distill? from noisy teacher to self-improvement. *arXiv preprint arXiv:2608.31046*, 2026.

[6] Sebastian Flennerhag, Yannick Schroecker, Tom Zahavy, Hado van Hasselt, David Silver, and Satinder Singh. Bootstrapped meta-learning. In *ICLR*, 2022.

[7] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. MiniLLM: Knowledge distillation of large language models. In *ICLR*, 2024.

[8] Suriya Gunasekar, Jason Lee, Daniel Soudry, and Nathan Srebro. Characterizing implicit bias in terms of optimization geometry. In *International Conference on Machine Learning*, pages 1832–1841. PMLR, 2018.

[9] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.

[10] Zhiwei He, Tian Liang, Jiahao Xu, Qiuzhi Liu, Xingyu Chen, Yue Wang, Linfeng Song, Dian Yu, Zhenwen Liang, Wenxuan Wang, et al. Deepmath-103k: A large-scale, challenging, decontaminated, and verifiable mathematical dataset for advancing reasoning. In *International Conference on Learning Representations*, volume 2026, pages 138306–138322, 2026.

[11] Byeongho Heo, Jaehui Hwang, Sangdoo Yun, and Dongyoon Han. On-policy delta distillation. *arXiv preprint arXiv:2607.15161*, 2026.

[12] Jonas Hübotter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*, 2026.

[13] Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. In *NeurIPS*, 2018.

[14] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025.

[15] Amirhossein Kazemnejad, Milad Aghajohari, Eva Portelance, Alessandro Sordoni, Siva Reddy, Aaron Courville, and Nicolas Le Roux. VinePPO: Refining credit assignment in RL training of LLMs. *arXiv preprint arXiv:2410.01679*, 2024.

[16] Yang Li, Semih Yavuz, and Shafiq Joty. RISE: Recursive improvement via self-extrapolating policy distillation. *arXiv preprint arXiv:2609.05295*, 2026.

[17] Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huan-ang Gao, Wenkai Yang, Zhiyuan Liu, and Ning Ding. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe, 2026.

[18] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let's verify step by step. In *ICLR*, 2024.

[19] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *NeurIPS*, 2023.

[20] Sagnik Mukherjee, Lifan Yuan, Pavan Jayasinha, Dilek Hakkani-Tür, and Hao Peng. Do we need adam? surprisingly strong and sparse reinforcement learning with sgd in llms. *arXiv preprint arXiv:2602.07729*, 2026.

[21] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *NeurIPS*, 2023.

[22] Rafael Rafailov, Joey Hejna, Ryan Park, and Chelsea Finn. From r to Q^*: Your language model is secretly a Q-function. In *COLM*, 2024.

[23] Yi Ren and Danica J. Sutherland. Learning dynamics of LLM finetuning. In *ICLR*, 2025.

[24] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[25] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 1279–1297, 2025.

[26] Wei Sun, Wen Yang, Pu Jian, Qianlong Du, Fuwei Cui, Shuo Ren, and Jiajun Zhang. KTAE: A model-free algorithm to key-tokens advantage estimation in mathematical reasoning. In *NeurIPS*, 2025.

[27] Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, Yuqiong Liu, An Yang, Andrew Zhao, Yang Yue, Shiji Song, Bowen Yu, Gao Huang, and Junyang Lin. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for LLM reasoning. *arXiv preprint arXiv:2506.01939*, 2025.

[28] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

[29] Wenkai Yang, Weijie Liu, Ruobing Xie, Kai Yang, Saiyong Yang, and Yankai Lin. Learning beyond teacher: Generalized on-policy distillation with reward extrapolation. *arXiv preprint arXiv:2602.12125*, 2026.

[30] Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *Advances in Neural Information Processing Systems*, 38:113222–113244, 2026.

[31] Lifan Yuan, Wendi Li, Huayu Chen, Ganqu Cui, Ning Ding, Kaiyan Zhang, Bowen Zhou, Zhiyuan Liu, and Hao Peng. Free process rewards without process labels. In *ICML*, 2025.

[32] Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*, 2026.

A Local derivation of the retained gradient

All derivatives are evaluated at θ_k on a fixed rollout batch. To make token selection explicit, let S denote the retained positions and let v_{it} denote the base distillation weights before masking. The virtual weights w_{it} and distillation weights v_{it} include their respective loss normalization and fixed importance corrections. Let J be the Jacobian of sampled-token log probabilities, with virtual weights W and retained weights V_S:

$$J_{it,:} = \boldsymbol{s}_{it}^\top, \qquad W = \mathrm{diag}(w_{it}), \qquad (V_S)_{it,it} = v_{it}\mathbf{1}\{(i,t) \in S\}. \tag{12}$$

Writing $a_{it} = A_i$ and collecting the log probabilities in ℓ,

$$g_k = J^\top W a, \qquad d = \ell(\theta_k + \alpha g_k) - \ell(\theta_k). \tag{13}$$

Expanding each log probability along the virtual step gives

$$d_{it} = \alpha \boldsymbol{s}_{it}^\top g_k + \frac{\alpha^2}{2} g_k^\top \nabla_\theta^2 \log \pi_{\theta_k}(x_{it} \mid c_{it}) g_k + O(\alpha^3). \tag{14}$$

The retained loss treats the teacher, targets, weights, and mask as fixed. Its negative gradient is therefore

$$h_k = J^\top V_S d = \alpha J^\top V_S J g_k + O(\alpha^2), \qquad F_{S,v} = J^\top V_S J. \tag{15}$$

The matrix F_k in the main text is $F_{S,v}$ in this explicit selection notation. Nonnegative retained weights make $F_{S,v}$ positive semidefinite. With full targets and matching stage weights, it has the empirical-Fisher form. Screening yields a selected, outcome-dependent score second moment, while the corresponding token-space transformation is determined by $JJ^\top$.

Its positive semidefiniteness gives the leading-order reward alignment

$$g_k^\top (\alpha F_{S,v} g_k) = \alpha \sum_{(i,t)\in S} v_{it} (\boldsymbol{s}_{it}^\top g_k)^2 \simeq \frac{1}{\alpha} \sum_{(i,t)\in S} v_{it} d_{it}^2 \geq 0. \tag{16}$$

Magnitude screening retains positions whose absolute advantage reaches the responsewise threshold τ_i, the $\lceil f|\mathcal{K}_i| \rceil$-th largest value in response i:

$$S_f = \{(i,t) : |d_{it}| \geq \tau_i\}. \tag{17}$$

Ties at the threshold are retained, so the selected fraction is nominal. The full-target setting uses $f = 1$.

Under the linear approximation, a fixed-cardinality subset maximizes alignment by retaining the largest $v_{it} d_{it}^2$, provided the base weights are fixed across candidate subsets. With fixed retention counts within each response and equal weights within that response, this criterion reduces to advantage magnitude. Unequal weights instead give the ranking $\sqrt{v_{it}}|d_{it}|$.

The Jacobian and score second moment appear only in the analysis. Training computes the targets through the temporary displacement and log-probability evaluation.

A.1 Stationary conditions under magnitude screening

Fix the rollout batch, outcome advantages, and virtual loss weights. The outcome gradient g_k lies in the span of the batch's token scores. Consider linearized targets $d_{it}^{\mathrm{lin}} = \alpha \boldsymbol{s}_{it}^\top g_k$, with $\alpha > 0$, and a retained set S containing at least one maximum-magnitude target from each response. With positive weight on each retained position, the corresponding direction satisfies

$$g_k^\top h_k^{\mathrm{lin}} = \alpha \sum_{(i,t)\in S} v_{it} (\boldsymbol{s}_{it}^\top g_k)^2. \tag{18}$$

If $h_k^{\text{lin}} = 0$, every retained score projection vanishes. Each response retains a maximum-magnitude projection, so all its remaining projections must also vanish. Thus g_k is orthogonal to the span containing it, implying $g_k = 0$. The converse is immediate, giving

$$h_k^{\text{lin}} = 0 \quad \Longleftrightarrow \quad g_k = 0 \tag{19}$$

for both full-target and responsewise magnitude-screened directions. This statement concerns the linearized teacher and raw surrogate gradients on a fixed batch. The implemented finite-displacement targets recover this construction to first order.

A.2 Sampling dependence from batch reuse

Batch reuse couples the reward gradient g_B and full-target score second moment F_B through the same sampled trajectories. Holding the actor fixed and writing $\bar{g} = \mathbb{E}[g_B]$ and $\bar{F} = \mathbb{E}[F_B]$, the expected first-order update decomposes as

$$\mathbb{E}[\alpha F_B g_B] = \alpha \bar{F}\bar{g} + \alpha \mathbb{E}[(F_B - \bar{F})(g_B - \bar{g})]. \tag{20}$$

Drawing an independent, identically distributed actor-rollout batch B' for distillation, with the realized teacher held fixed, removes the covariance term and gives $\mathbb{E}[\alpha F_{B'} g_B] = \alpha \bar{F}\bar{g}$. The difference between these expectations captures the sampling dependence introduced by reuse.

The same coupling makes teacher construction sensitive to batch partitioning. Divide fixed rollouts into M parts, preserving complete prompt groups and their advantages. At a common actor, let g_j and F_j be each part's reward gradient and full-target score moment, normalized so the pooled quantities are their arithmetic means g_{pool} and F_{pool}. With a common SGD lookahead rate, the linearized raw directions are

$$h_{\text{pooled}} = \alpha F_{\text{pool}} g_{\text{pool}}, \qquad h_{\text{separate}} = \frac{\alpha}{M} \sum_{j=1}^{M} F_j g_j. \tag{21}$$

Their difference is

$$h_{\text{separate}} - h_{\text{pooled}} = \frac{\alpha}{M} \sum_{j=1}^{M} (F_j - F_{\text{pool}})(g_j - g_{\text{pool}}). \tag{22}$$

The pooled product includes terms $F_j g_l$ for $j \neq l$, exposing each part's contexts to reward evidence from other parts. Separate teachers omit these interactions. This identity needs no independence assumption and compares raw directions before the real optimizer acts. It assumes consistent loss normalization and no separate gradient clipping or adaptive rescaling within parts. Accumulating the reward gradient before constructing a common teacher, then accumulating its distillation gradients before one actor update, recovers the pooled procedure.

A.3 Directional reweighting of the reward gradient

Let $F_k u_j = \lambda_j u_j$, where the u_j form an orthonormal basis and $\lambda_j \geq 0$. Projecting the local gradient relation onto each eigenvector gives

$$u_j^\top h_k \approx \alpha \lambda_j \, u_j^\top g_k. \tag{23}$$

The ratio of any two nonzero gradient components is multiplied by the corresponding eigenvalue ratio. High-sensitivity modes therefore receive greater relative weight, even if the total gradient norm contracts. An isotropic F_k reduces the transformation to scalar rescaling. These modes belong to the eigensystem of F_k, distinct from the singular spectra of reshaped parameter-gradient matrices.

A.4 Geometry of the virtual update

Changing the virtual optimizer changes the displacement that enters distillation. A teacher constructed along u_k induces

$$h_k \approx \alpha F_k u_k. \tag{24}$$

If $u_k = P_k g_k$, the resulting transformation is $\alpha F_k P_k g_k$. The SGD lookahead uses $P_k = I$ and gives the forward score-moment transformation studied in the main text.

Empirical natural-gradient preconditioning with the same score moment provides a useful limiting case [2]. With full targets, positive score weights, and every score contributing to g_k represented in F_k, the outcome gradient lies in the range of F_k. Its Moore–Penrose inverse $F_k^\dagger$ then gives

$$u_k = F_k^\dagger g_k \quad \Longrightarrow \quad h_k \approx \alpha F_k F_k^\dagger g_k = \alpha g_k. \tag{25}$$

Policy-space distillation thus cancels the inverse-metric transformation locally when both stages use the same score moment. A screened or otherwise mismatched distillation operator generally produces a different direction.

A.5 The token-local distillation gradient

Differentiating the conditional reverse KL with the teacher and prefix held fixed yields

$$\nabla_\theta \, \mathrm{KL}(\pi_\theta \| \pi_E) = \mathbb{E}_{a \sim \pi_\theta} \left[\left(\log \frac{\pi_\theta(a \mid c)}{\pi_E(a \mid c)} + 1 \right) \nabla_\theta \log \pi_\theta(a \mid c) \right]. \tag{26}$$

The expected score vanishes, yielding the teacher–actor log ratio as the ascent coefficient in Equation equation 4, with the prefix distribution held fixed. In OPGD, the teacher depends on the sampled batch and the resulting advantages are detached. Equation equation 7 defines the finite-batch surrogate, differentiated through the restored actor's log probabilities.

A.6 Teacher extrapolation and virtual step size

The extrapolated teacher admits an exact exponential-tilting form. With $\gamma = 1 + \omega$ and a common temperature for actor and teacher logits, Equation equation 8 defines

$$\begin{aligned} q_\gamma(a \mid c) &= \mathrm{softmax}\big(\gamma z_E(c) + (1-\gamma) z_k(c)\big)_a \\ &= \frac{\pi_{\theta_k}(a \mid c) \exp\{\gamma d(c,a)\}}{Z_\gamma(c)}, \qquad Z_\gamma(c) = \mathbb{E}_{a \sim \pi_{\theta_k}(\cdot \mid c)} \left[\exp\{\gamma d(c,a)\} \right]. \end{aligned} \tag{27}$$

Its log ratio to the actor is therefore

$$\log \frac{q_\gamma(a \mid c)}{\pi_{\theta_k}(a \mid c)} = \gamma d(c,a) - \log Z_\gamma(c). \tag{28}$$

Holding the teacher and prefix distribution fixed, the full-action OPD direction at the original actor is

$$\begin{aligned} h_\gamma &= \mathbb{E}_{c, a \sim \pi_{\theta_k}} \left[\big(\gamma d(c,a) - \log Z_\gamma(c) \big) \nabla_\theta \log \pi_{\theta_k}(a \mid c) \right] \\ &= \gamma \, \mathbb{E}_{c, a \sim \pi_{\theta_k}} \left[d(c,a) \nabla_\theta \log \pi_{\theta_k}(a \mid c) \right] = \gamma h_1. \end{aligned} \tag{29}$$

The prefix-dependent normalizer cancels because the conditional expected score is zero, making this scaling exact for any fixed teacher, independently of the virtual step size.

Relating this extrapolation to a larger parameter step requires a local approximation. As $\alpha \to 0$ with γ fixed, the score identity gives $\mathbb{E}_a[d(c,a)] = O(\alpha^2)$ and $\log Z_\gamma(c) = O(\alpha^2)$, yielding

$$\log \frac{q_\gamma(a \mid c)}{\pi_{\theta_k}(a \mid c)} = \gamma\alpha \nabla_\theta \log \pi_{\theta_k}(a \mid c)^\top g_k + O(\alpha^2), \tag{30}$$

which agrees to first order with the log-probability change under a virtual step of size $\gamma\alpha$. Parameter-space extrapolation gives the corresponding identity exactly:

$$\theta_k + \gamma(\theta_k^E - \theta_k) = \theta_k + \gamma\alpha g_k. \tag{31}$$

At finite step size, network curvature separates logit extrapolation from a scaled parameter step. The prefix-dependent normalizer cancels under a fixed-teacher, full-action score expectation, but need not cancel in the finite-batch screened surrogate. The local equivalence requires both the original and scaled teacher displacements to remain small.

B Qwen3-8B Evaluation

Table 5 presents the detailed Qwen3-8B evaluation results across different sampling budgets. We report Pass@k for $k \in \{1, 2, 4, 8\}$ on four in-domain mathematical reasoning benchmarks. Due to computational constraints, we sample only four responses per problem on the five out-of-domain code-generation benchmarks, and therefore report Pass@k for $k \in \{1, 2, 4\}$.

Across all four mathematical benchmarks, OPGD consistently outperforms GRPO at every evaluated sampling budget. It also achieves higher average Pass@k on the code-generation benchmarks, suggesting that the improvements extend beyond the training domain.

Table 5: Detailed Qwen3-8B evaluation results. Both GRPO and OPGD are evaluated after 100 training updates, while Base denotes the initial model before training. Mathematical reasoning benchmarks are in-domain, whereas code-generation benchmarks evaluate out-of-domain transfer. All values are percentages. Due to computational constraints, code-generation Pass@8 is not evaluated. Boldface indicates the best result in each column within a Pass@k block (including ties), and blue shading highlights OPGD.

Metric	Method	Math (In-domain)					Code (Out-of-domain)					
		AIME24	AIME25	HMMT-Feb	HMMT-Nov	avg4	HumanEval	HumanEval+	MBPP	MBPP+	LCB v6	avg5
Pass@1	Base	27.2	21.7	10.1	9.2	17.0	88.0	81.3	80.8	69.2	17.0	67.2
	GRPO	54.0	41.0	26.0	35.6	39.2	85.1	80.5	81.5	70.8	17.4	67.1
	OPGD	**67.2**	**51.2**	**31.9**	**44.1**	**48.6**	**88.7**	**82.2**	**84.8**	**73.3**	**18.1**	**69.4**
Pass@2	Base	35.7	26.9	13.4	14.8	22.7	**92.9**	86.9	86.7	75.0	20.1	72.3
	GRPO	65.8	50.4	33.2	44.4	48.5	87.9	84.2	86.6	75.3	20.1	70.8
	OPGD	**75.6**	**61.4**	**40.8**	**51.8**	**57.4**	92.5	**87.4**	**88.1**	**76.7**	**20.5**	**73.0**
Pass@4	Base	43.5	31.6	16.9	21.7	28.4	**95.7**	89.6	89.4	77.8	**22.9**	75.1
	GRPO	74.8	58.8	40.2	50.9	56.2	90.2	86.6	89.2	77.8	**22.9**	73.3
	OPGD	**79.8**	**69.4**	**48.8**	**58.6**	**64.1**	94.5	**90.9**	**90.0**	**79.4**	**22.9**	**75.5**
Pass@8	Base	50.3	36.9	20.7	29.3	34.3	–	–	–	–	–	–
	GRPO	80.2	65.9	45.8	56.4	62.1	–	–	–	–	–	–
	OPGD	**81.4**	**75.0**	**54.6**	**65.1**	**69.0**	–	–	–	–	–	–

C Mathematical evaluation

Given N problems, n sampled completions per problem, and c_j correct completions for problem j, we estimate Pass@k as [3]

$$\widehat{\text{Pass@}k} = \frac{1}{N}\sum_{j=1}^{N}\left[1 - \frac{\binom{n-c_j}{k}}{\binom{n}{k}}\right]. \tag{32}$$

For $k = 1$, this reduces to mean sample correctness.

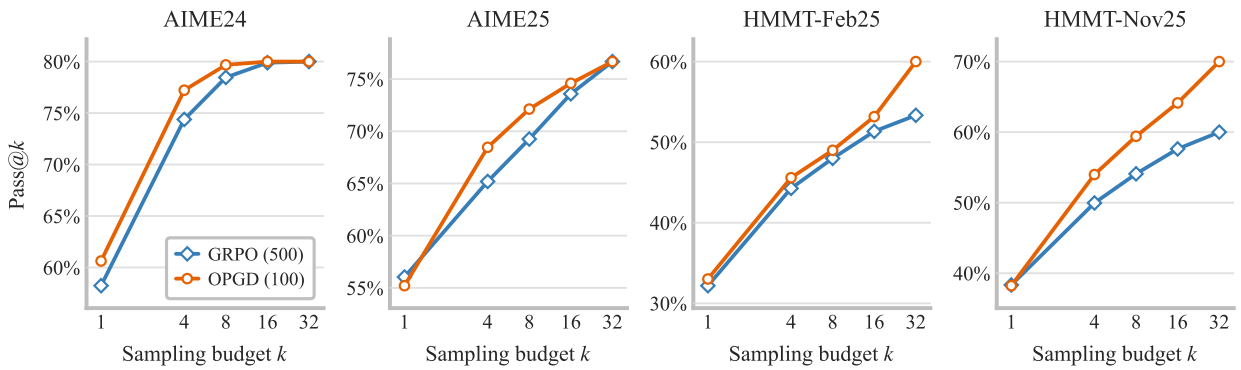


Figure 7: Pass@k against the sampling budget k on a $\log_2$ axis, for full-target OPGD in 100 updates and the 500-update GRPO reference from Yang et al. [29], on the four benchmarks of Table 1.

Table 6 gives the complete version of Table 1, with all three Pass@k metrics and the per-benchmark breakdown for every configuration.

Table 6: Complete version of Table 1: mathematical reasoning performance. The 500-update GRPO reference uses published weights from Yang et al. [29]

Metric	Method	Budget	AIME24	AIME25	HMMT-Feb25	HMMT-Nov25	Mean
Pass@1	Qwen3-4B	–	0.2229	0.2073	0.1094	0.0875	0.1568
	GRPO	100	0.4917	0.4281	0.2688	0.3135	0.3755
	GRPO (entropy 20%)	100	0.5458	0.4760	0.2833	0.3385	0.4109
	DAPO	100	0.5531	0.5240	0.2844	0.3552	0.4292
	GRPO	200	0.5844	0.5115	0.2969	0.3469	0.4349
	GRPO (external)	500	0.5823	**0.5604**	0.3219	0.3833	0.4620
	OPGD (full)	100	0.6063	0.5521	0.3302	0.3823	0.4677
	OPGD (top 10%)	100	**0.6667**	0.5448	**0.3542**	**0.3906**	**0.4891**
Pass@4	Qwen3-4B	–	0.3656	0.2881	0.1907	0.1954	0.2600
	GRPO	100	0.7184	0.6183	0.3933	0.4761	0.5515
	GRPO (entropy 20%)	100	0.7259	0.6278	0.4168	0.4805	0.5628
	DAPO	100	0.7277	0.6575	0.4085	0.4855	0.5698
	GRPO	200	0.7486	0.6375	0.4425	0.4909	0.5799
	GRPO (external)	500	0.7438	0.6519	0.4427	0.4996	0.5845
	OPGD (full)	100	0.7722	0.6846	0.4559	**0.5399**	0.6132
	OPGD (top 10%)	100	**0.8122**	**0.6921**	**0.4793**	0.5392	**0.6307**
Pass@8	Qwen3-4B	–	0.4333	0.3339	0.2323	0.2745	0.3185
	GRPO	100	0.7843	0.6697	0.4483	0.5199	0.6056
	GRPO (entropy 20%)	100	0.7784	0.6787	0.4755	0.5398	0.6181
	DAPO	100	0.7736	0.6902	0.4649	0.5327	0.6154
	GRPO	200	0.7853	0.6804	0.5106	0.5452	0.6304
	GRPO (external)	500	0.7846	0.6925	0.4799	0.5409	0.6245
	OPGD (full)	100	0.7968	0.7212	0.4899	**0.5941**	0.6505
	OPGD (top 10%)	100	**0.8308**	**0.7354**	**0.5329**	0.5885	**0.6719**

D Training configuration

Table 7 reports the configuration of the in-house mathematical training runs.The training framework is built on VERL [25]. Validation runs every 10 updates on AIME'24 and AIME'25, and the checkpoint reported for each in-house run and update budget in Table 1 is the one with the highest mean Pass@1 over the two. All results reported in the tables are produced with the official evaluation scripts from the G-OPD repository [29]. Those scripts run outside the VERL training framework, and their scores can differ slightly from the in-training evaluation for infrastructure reasons.

Table 7: Mathematical training configuration. All settings are shared by the in-house mathematical runs except the temporary SGD step, which applies only to OPGD.

Setting	Value
Base model	Qwen3-4B
Prompt batch size / responses per prompt	128 / 8
Maximum prompt / response length	2048 / 16,384
Sampling temperature / top-p	1.0 / 1.0
Optimizer	AdamW, $\beta = (0.9, 0.999)$
Learning rate	10^{-6}, constant
Gradient clipping / weight decay	1.0 / 0.01
Minibatch / epochs	128 / 1
PPO clip ratio	0.2
Importance correction	token-level, clip 5
KL / entropy coefficient	0.0 / 0.0
Advantage normalization	group standard deviation
Virtual step (OPGD only)	SGD, $\alpha = 0.1$

Table 8 reports the configuration of the in-house code training runs.

Table 8: Mathematical training configuration. All settings are shared by the in-house code runs except the temporary SGD step, which applies only to OPGD.

Setting	Value
Base model	Qwen3-4B
Prompt batch size / responses per prompt	128 / 8
Maximum prompt / response length	2048 / 8,192
Sampling temperature / top-p	1.0 / 1.0
Optimizer	AdamW, $\beta = (0.9, 0.999)$
Learning rate	10^{-6}, constant
Gradient clipping / weight decay	1.0 / 0.01
Minibatch / epochs	128 / 1
PPO clip ratio	0.2
Importance correction	token-level, clip 5
KL / entropy coefficient	0.0 / 0.0
Advantage normalization	group standard deviation
Virtual step (OPGD only)	SGD, $\alpha = 0.1$

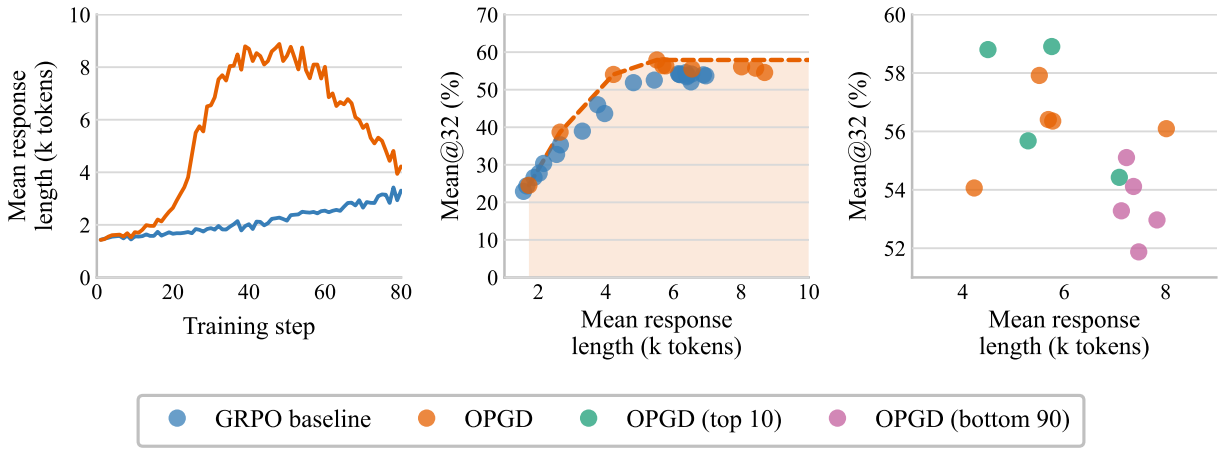


Figure 8: Response length and checkpoint accuracy. Left: mean response length during full-target OPGD and GRPO training. Middle: checkpoint accuracy against mean response length for these runs. Right: full-target, top-10%, and bottom-90% OPGD checkpoints after update 50.

E Training dynamics and computational cost

Full-target OPGD spans a wide range of response lengths after reaching its AIME accuracy plateau, whereas GRPO gains length and accuracy together (Figure 8). Among the target-subset variants, the later top-10% checkpoints combine higher accuracy with shorter responses (Figure 6).

These length changes affect generation cost alongside the teacher-construction overhead profiled in Figure 4. Teacher scoring and contextual backpropagation process the complete sampled sequences even when the loss retains a subset of token targets, so their costs do not decrease in proportion to the selected fraction.

F Code-generation experiments

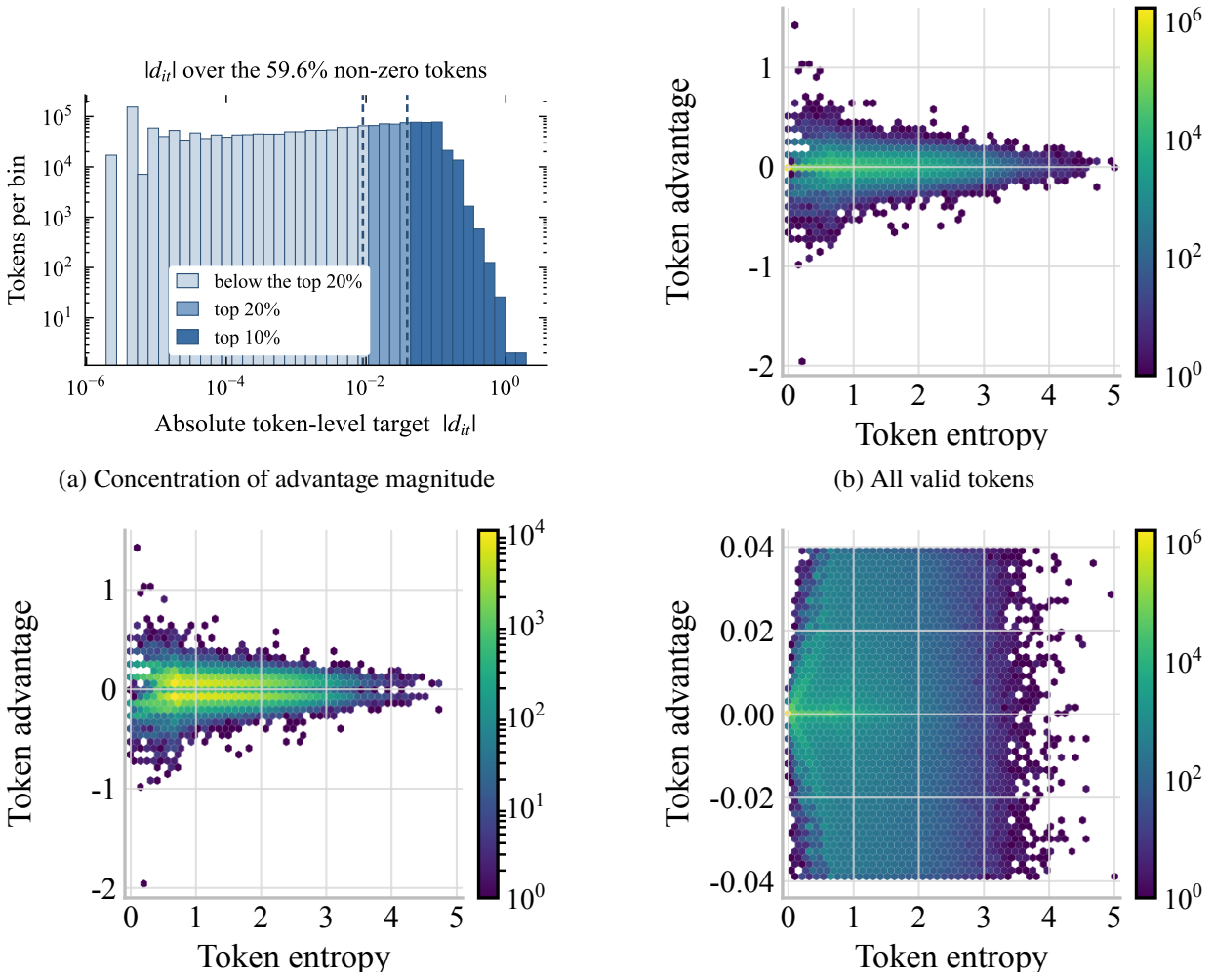
Both methods are trained separately on coding prompts. We evaluate their checkpoints on HumanEval+ and MBPP+ [19], and LiveCodeBench v6 [14], using four completions per problem every ten updates through update 100. OPGD exceeds GRPO at every shared checkpoint on all three benchmarks (Figure 3). Table 9 summarizes the gains and the first evaluated OPGD update reaching GRPO's update-100 score.

Table 9: Results from separate code-generation training runs. Pass@1 gains over GRPO are in percentage points. The mean gain averages the ten shared checkpoints through update 100. The last column gives the first evaluated OPGD update reaching GRPO's update-100 score.

Benchmark	Mean gain	Gain at update 100	First matching update
HumanEval+	+3.6	+3.3	40
MBPP+	+2.3	+1.4	30
LiveCodeBench v6	+3.1	+3.7	40

G Target distributions

On the first diagnostic batch, 40.4% of valid positions have zero sampled-token log-probability difference at the measurement precision. Figure 9 plots the joint distribution of targets and token entropy, partitioned by the batch-global magnitude threshold. Absolute target mass describes the concentration of the loss coefficients. Their gradient contributions also depend on token score vectors and loss normalization.



(a) Concentration of advantage magnitude

(b) All valid tokens

(c) Global top 10%

(d) Complementary 90%

Figure 9: OPD advantages on the first diagnostic rollout batch. (a) Distribution of nonzero magnitudes, with batch-global cutoffs. The top 10% and 20% carry 76.9% and 95.4% of absolute mass, respectively. (b–d) Signed advantages against token entropy for all positions and the two subsets defined by the global 90th-percentile magnitude threshold. Color denotes log token counts on separate scales. The complementary 90% uses a narrower vertical range.

H Tokens without an outcome gradient

A prompt group whose verifier rewards agree contributes nothing to the outcome objective: the within-group mean equals every reward, so GRPO assigns each token of each response in that group a coefficient of zero, whether the responses are all correct or all incorrect. Figure 10 measures how much of the first diagnostic batch falls into the two families, and how much of it GRPO therefore generates, scores, and then discards.

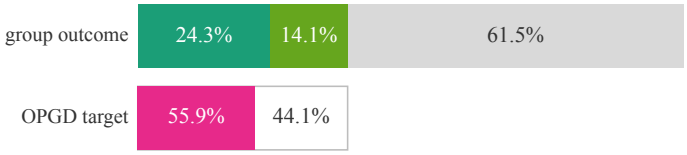


Figure 10: **First-step rollout tokens.** Greens: groups with uniform rewards, where GRPO has no advantage. Below: the share with a nonzero OPGD target.

OPGD keeps that share. Its token-level target comes from the reward-directed update inside the teacher, which is computed over the whole batch, so a group with no outcome gradient of its own still receives a preference over its tokens from the groups that have one. The lower row of Figure 10 counts a separate property, whether a token carries a nonzero teacher target, and it is nonzero on most of the tokens in the two uniform-reward families, with both signs. These coefficients carry the contextual credit of Appendix J.

The two objectives therefore differ in what they extract from a batch of a given size, not in the batch itself. The tokens GRPO discards are where part of the per-update teacher overhead of Appendix E is spent.

I Gradient structure

A shared-batch diagnostic compares the outcome and full-target OPGD surrogate gradients at the same parameters. Of the 398 parameter tensors, 253 are matrices: seven weight matrices in each of 36 transformer blocks and the embedding. Spectral measurements use these matrices, excluding the 145 vectors with trivial singular spectra. The 37 plotted groups comprise the transformer blocks and the embedding, with block statistics computed as means of the per-matrix quantities.

For each gradient matrix G, a randomized rank-16 factorization estimates the leading singular values. The normalization uses the exact squared Frobenius norm $E = \|G\|_F^2$. With estimated leading masses $p_j = \widehat{\sigma}_j^2/E$ and residual mass $q = 1 - \sum_{j=1}^{16} p_j$, the spectral entropy proxy assigns the residual uniformly over the remaining $r - 16$ directions:

$$\widehat{H} = -\sum_{j=1}^{16} p_j \log p_j - q \log \frac{q}{r-16}, \qquad r = \min(\mathrm{rows}(G), \mathrm{cols}(G)). \tag{33}$$

Exponentiating this proxy gives the effective rank under the flat-tail approximation. These spectra describe individual parameter-gradient matrices; the operator in Equation equation 10 acts on the full parameter vector.

The leading direction and the leading sixteen-dimensional subspace account for more energy under distillation in nearly all matrices (Figures 11 and 5). The accompanying entropy reduction extends across the layer groups (Figure 12), with substantial variation among projection types. Attention output and MLP down projections, which write into the residual stream, have the largest reductions in mean estimated effective rank, while attention key projections have the smallest. Three key projections, at layers 10, 11, and 23, increase in estimated effective rank even though mean spectral entropy decreases in every layer group.

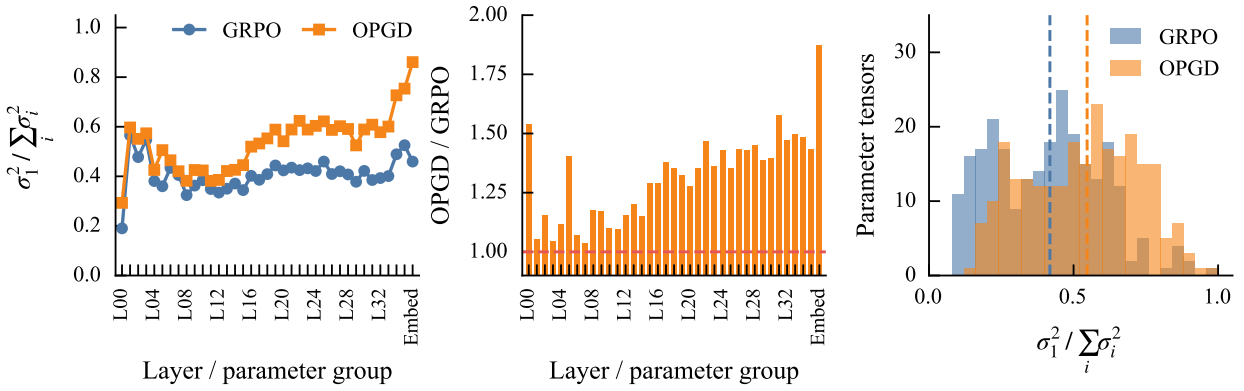


Figure 11: Estimated singular-value concentration of gradient matrices on a shared diagnostic batch. Concentration is the leading squared singular value divided by the exact squared Frobenius norm. Left: layer-group averages of the per-matrix statistic. Middle: corresponding OPGD-to-GRPO ratios. Right: the distribution over 253 matrix-shaped parameter tensors.

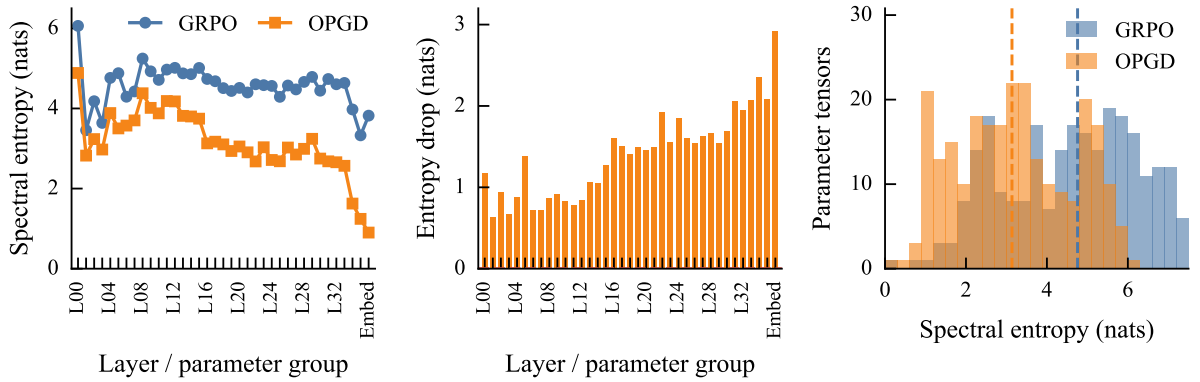


Figure 12: Spectral-entropy proxy for the same diagnostic gradients. Left: layer-group means over the matrix-shaped parameter tensors. Middle: the reduction under OPGD. Right: the distributions over all 253 matrices, with medians dashed. The estimator uses the leading sixteen estimated singular values and a uniform residual tail.

The full-target gradient has a smaller norm in every layer group while remaining positively aligned with the outcome gradient (Figure 13). Fewer coordinates exceed the fixed absolute threshold. This fraction depends on gradient scale, whereas the normalized spectral measures above capture changes that persist under a common rescaling.

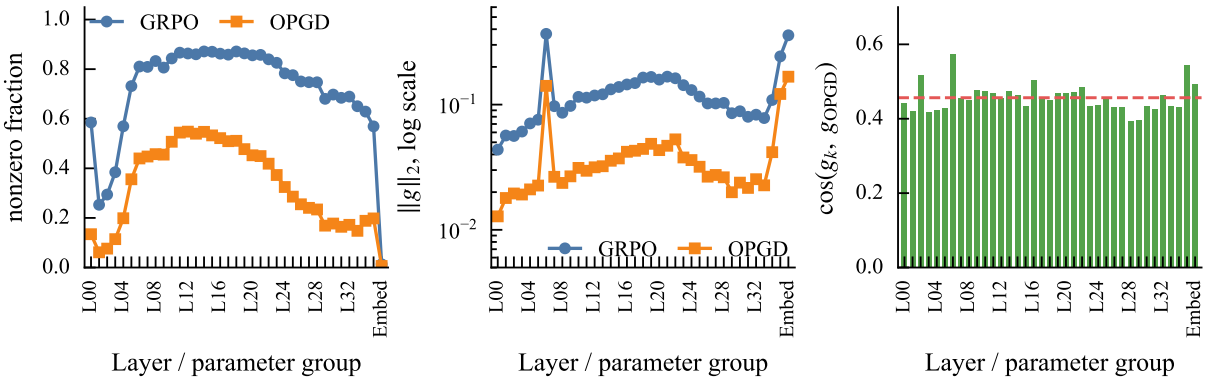


Figure 13: Magnitude and alignment of the same diagnostic gradients across layer groups. Left: fraction of coordinates exceeding 10^{-6} in absolute value. Middle: L2 norm on a logarithmic axis. Right: cosine between the outcome and full-target OPGD gradients.

J Contextual credit across responses

Figures 14 and 15 extend the repeated-word example to six response fragments and six complete responses. Both signs appear inside trajectories with positive, negative, and zero group advantages, so the response coefficient does not determine the sign pattern of the teacher's preferences. Contextual feedback also reaches trajectories whose own prompt group contributes no outcome gradient, through the reward-directed update from other groups. Color saturation is set by the batch-global 90th percentile of absolute advantage, distinct from the responsewise training threshold.

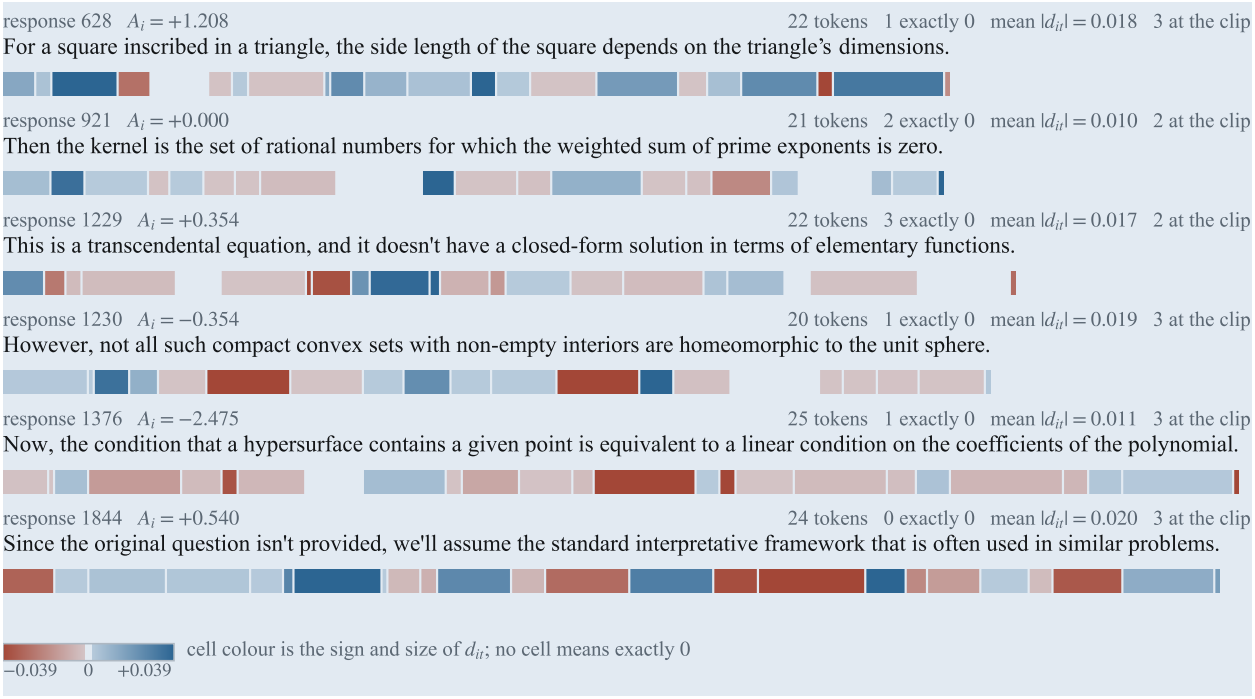


Figure 14: Sampled-token OPD advantages in six response fragments spanning positive, zero, and negative group advantages. Blue denotes positive log differences, red denotes negative differences, and blank cells denote stored zeros. Recorded summary statistics accompany each fragment.



Figure 15: OPD advantages across six complete responses, ordered by increasing group advantage. Cells represent tokens in reading order, with a fixed number per line. Both signs and varying magnitudes occur throughout each response.

Table 10 quantifies the complete-response maps. Responses with matched absolute group advantage can have substantially different proportions of zero and large-magnitude targets. This variation is consistent with the dependence of contextual feedback on token-score alignment, rather than on the trajectory coefficient alone.

Table 10: Statistics of the six complete responses in Figure 15, in the same order. Zero targets are measured at the stored precision. The last column is the fraction at or above the batch-global 90th-percentile magnitude cutoff, rather than the responsewise threshold used for training. Mean absolute target magnitude includes all valid response tokens.

Response	A_i	Tokens	Zero targets	Mean $\lvert d_{it} \rvert$	At/above cutoff
1696	−2.47	1739	32%	0.0148	13%
1456	−0.94	1397	53%	0.0073	6%
182	−0.35	1345	20%	0.0175	15%
329	+0.35	1306	42%	0.0104	8%
1091	+0.94	1238	49%	0.0095	8%
1986	+2.47	1211	14%	0.0188	18%